Assignment #9 PDF code:

```
# %% [markdown]
# Nair-Trishla-Assignment-9-Notebook

## Imports:
# %% [markdown]
Vega-Lite:
# %% [javascript]
await import("https://cdn.jsdelivr.net/npm/vega@5.20.2/build/vega.min.js");
await import("https://cdn.jsdelivr.net/npm/vega-lite@5.1.0/build/vega-lite.min.js");
await import("https://cdn.jsdelivr.net/npm/vega-embed@6.18.2/build/vega-embed.min.js");
# %% [javascript]
vegaEmbed
# %% [markdown]
Python:
# %% [python]
import pyodide
import matplotlib
# %% [python]
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
# %% [markdown]
## Visuals:
# %% [python]
URL = 'https://raw.githubusercontent.com/UIUC-iSchool-
DataViz/is445_bcubcg_fall2022/main/data/building_inventory.csv'
df = pd.read_csv(pyodide.open_url(URL))
```

```
df
# %% [python]
sdf = df[['County', 'City']]
#df1 = sdf.groupby('City')[['County']].nunique()
df1 = sdf.groupby(['City']).agg("count").reset_index()
df1
# %% [python]
city = df['City'].values.tolist()
county = df1['County'].values.tolist()
# %% [javascript]
cityJs = pyodide.globals.get("city");
cityJs = cityJs.toJs();
# %% [javascript]
countyJs = pyodide.globals.get("county");
countyJs = countyJs.toJs();
# %% [html]
<div id="countiesFromPython"></div>
# %% [javascript]
var myBar1Spec = {
  "data":{"values":[
    {"Illinois Cities":cityJs, "Buildings in Counties":countyJs}
  ]},
  "transform":[{"flatten":["Illinois Cities","Buildings in Counties"]}],
  "mark": { "type": "bar", "cornerRadiusEnd": 5},
  "height": "500",
  "width": "1000",
  "encoding":{
    "x":{"field":"Illinois Cities", "type": "nominal"},
    "y":{"field":"Buildings in Counties", "type": "quantitative"}
```

```
 }
};

var v = vegaEmbed("#countiesFromPython",myBar1Spec)
# %% [python]
df2 = df[['Bldg Status', 'Year Constructed', 'Year Acquired']]
df2
# %% [python]
value1 = df2['Bldg Status'].values.tolist()
value2 = df2['Year Constructed'].values.tolist()
value3 = df2['Year Acquired'].values.tolist()
# %% [javascript]
statusJs = pyodide.globals.get("value1");
statusJs = statusJs.toJs();
# %% [javascript]
constructJs = pyodide.globals.get("value2");
constructJs = constructJs.toJs();
# %% [javascript]
acquiredJs = pyodide.globals.get("value3");
acquiredJs = acquiredJs.toJs();
# %% [html]
<div id="buildingstatusFromPython"></div>
# %% [javascript]
var bar2Spec = {
  "data":{"values":[
    {"Year Acquired":acquiredJs,  "Year Constructed":constructJs,  "Building Status": statusJs}
  ]},
  "transform":[{"flatten":["Year Acquired","Year Constructed","Building Status"]}],
  "mark":"point",
```

```
 "height": "500",
 "width": "1000",
 "encoding":{
  "x":{"field":"Year Acquired", "type": "nominal"},
  "y":{"field":"Year Constructed", "type": "nominal"},
  "color": {"field": "Building Status", "type": "nominal", "scale": {"range": ["#5c32a8", "#32e6a4",
"#a64505"]}},
  "shape": {"field": "Building Status", "type": "nominal"}
 }
};

var v = vegaEmbed("#buildingstatusFromPython", bar2Spec)
```

%% [markdown]

Write-Up:

%% [markdown]

The first visualization I made depicts the number of buildings in the counties, per city. Originally, my intended data was supposed to show the number of counties in a city in Illinois but for some reason the data wasn't shown correctly. The best way I felt to show the current data was as a bar chart because it made the most sense when dealing with one nominal type and one quantitative type. With this visual, I wanted to highlight the Vega-lite feature where you can alter the shape of the bars in a bar graph and thus I left the visual simple and didn't do much with the colors and changed the graphs size to fit the page. If I had more time I would fix the data inputted into the visual so that it makes more sense and accomplish what wish to present initially.

%% [markdown]

The second visual I made looks at the buildings and the year they were acquired and the year they were created. With this one, I wanted to focus on adding various colors and shapes on a scatterplot. So, while the shapes for the points are the default shapes available, the colors were chosen with their Hex numbers. As for the axes, though the x is correct, the y axis goes from top to bottom rather than bottom to top for some reason. For the types of data, I left it as nominal rather than the date type because it is just the year. If I was given more time I would try to add maybe add a regression line and fix the y axis's orientation.